

De una barbería a muchas

Qué hace falta para que esta app deje de ser de un negocio y pase a ser de varios, sin que ninguno vea la plata ni los clientes del otro.

23 tablas hoy 2 sedes • 6 barberos 65 migraciones aplicadas 23 de agosto de 2026

Sí se puede, y estás a mitad de camino sin haberlo buscado.

La app ya piensa por *sedes*: casi todo lo operativo está acotado a un local y hay guardas reales que lo hacen cumplir. Multitenencia es exactamente el mismo patrón una capa más arriba. Lo difícil no es agregar la columna — es que la base defienda el límite sola, sin depender de que el código se acuerde.

Pero no lo hagas todavía. Con dos o tres clientes sale más barato y más seguro clonar el despliegue. Este esquema es para cuando quieras que alguien se registre desde la web, o cuando aplicar migraciones a mano en N proyectos ya sea un trabajo. Mientras tanto, la fase 0 —que es puro orden— hace que el día que decidas, sea barato.

01 El modelo

Una base para todos, con la frontera puesta en el motor.

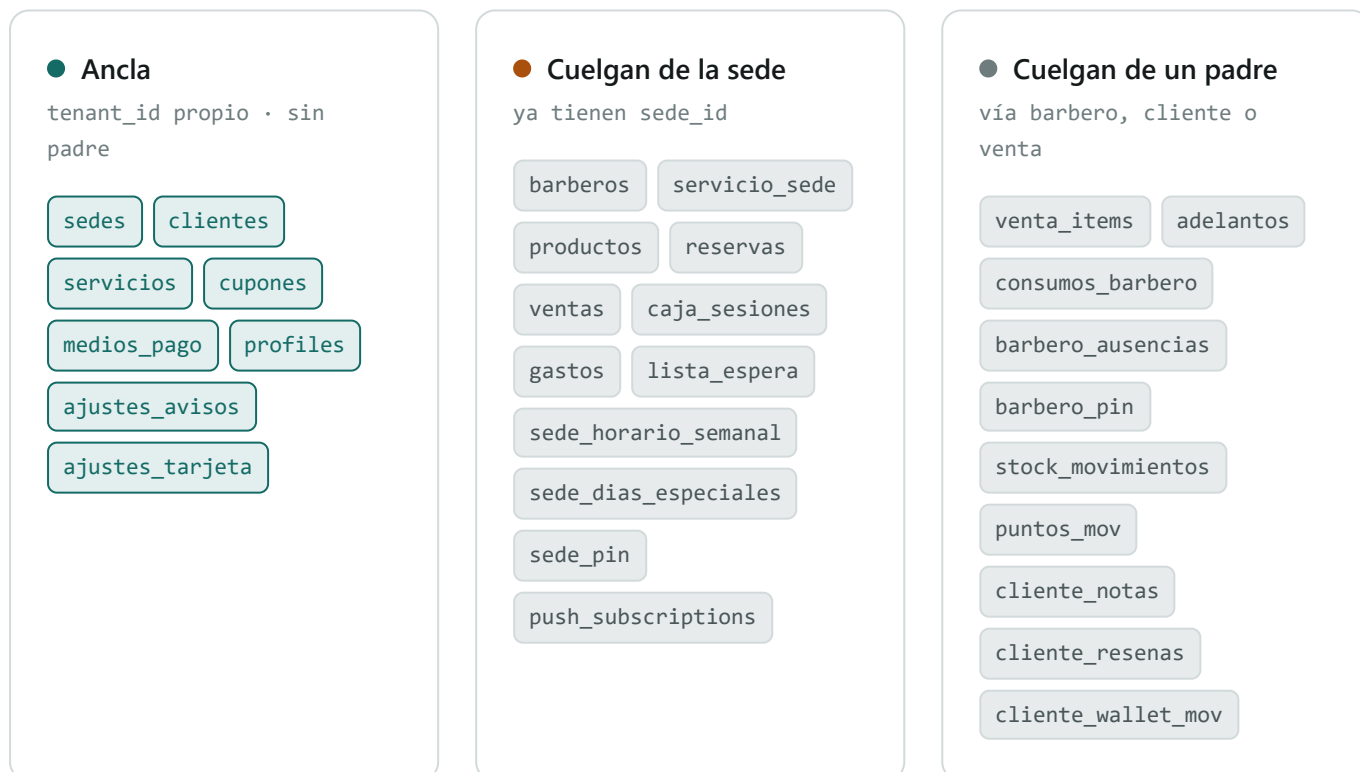
Hay dos formas de hacer esto y la elección define todo lo demás.

CAMINO	A FAVOR	EN CONTRA
Un despliegue por cliente <i>clonar lo que ya tenés</i>	Imposible que se filtren datos entre negocios. Arranca en días. Cada uno con su marca y su dominio sin pelear.	N proyectos, N migraciones, N configuraciones de correo. A partir de ~10 es un trabajo de tiempo completo.
Una base compartida <i>lo que describe este documento</i>	Una migración sirve para todos. Registro propio posible. Costo por cliente casi cero.	Un error en una política de seguridad es una fuga entre negocios. No perdona.

Si vas a compartir base, la regla es una sola y no se negocia: **la frontera vive en Postgres, no en el código**. El día que alguien escriba una consulta sin el filtro —y va a pasar— la base tiene que devolver cero filas, no los clientes del vecino.

02 Dónde va el dueño

Las 23 tablas, agrupadas por cómo saben a qué negocio pertenecen.



La tentación es dejar que los dos últimos grupos deduzcan el dueño por su padre — un `JOIN` hasta llegar a `sedes.tenant_id`. **No lo hagas**. Una política de seguridad con un `JOIN`

adentro se ejecuta por cada fila leída, y basta un `LEFT JOIN` mal puesto para que una fila huérfana se escape del filtro.

Poné `tenant_id` en las 23. Suena redundante y lo es — a propósito. A cambio, toda política del sistema es la misma línea, usa un índice, y no hay que razonar sobre nada:

```
create policy aisla_por_negocio on public.ventas
  for all to authenticated
  using (tenant_id = public.tenant_actual())
  with check (tenant_id = public.tenant_actual());
```

Y el `tenant_id` no lo manda la app: lo pone la base sola, así que ninguna inserción puede olvidarse de él.

```
alter table public.ventas
  add column tenant_id uuid not null
  default public.tenant_actual()
  references public.tenants(id);
```

03 Quién pregunta

Dos caminos entran a la base, y solo uno tiene sesión.

`tenant_actual()` resuelve el negocio del que pregunta. Para el **staff** sale de su perfil, y conviene fijarlo como *claim* en el token para no pagar una consulta por cada fila:

```
create function public.tenant_actual() returns uuid
language sql stable security definer set search_path = ''
as $$
  select nullif(auth.jwt() -> 'app_metadata' ->> 'tenant_id', '')::uuid
$$;
```

El **cliente que reserva** es otra historia, y acá está el punto que más se subestima: **el sitio público escribe con `service_role`, que se salta la RLS por diseño**. Ahí la base no te protege — el dueño lo decide el dominio por el que entró y lo impone el código.

Por eso el `tenant_id` de la reserva pública tiene que resolverse en un *único* lugar —el middleware que ya existe en `proxy.ts`, mapeando el host a su negocio— y viajar desde ahí. Ninguna acción pública debería aceptar un `tenant_id` que venga del navegador.

QUIÉN ENTRA	CÓMO SE SABE EL NEGOCIO	QUÉ LO PROTEGE
Dueño y barberos	claim del token	RLS
Cliente con cuenta	claim del token	RLS
Cliente reservando	dominio → middleware	solo el código
Los crones de correo	recorren todos los negocios	ninguno: son de servicio

04 Las tres trampas

Están en el esquema de hoy y ninguna avisa cuando explota.

Los identificadores son palabras, no números

`sedes.id` y `servicios.id` son texto: `'parque-venezuela'`, `'cerquillo'`. Funciona hermoso con un negocio y revienta con dos: **toda barbería quiere un servicio llamado `corte`**, y la segunda que lo intente choca contra la llave primaria.

Se arregla de una de dos formas: llave compuesta `(tenant_id, id)` —barata, pero contagia el `tenant_id` a cada clave foránea— o pasar a `uuid` y dejar el slug como columna de display. **La segunda es la correcta y hay que hacerla antes de tener datos de varios negocios**, porque después es una migración con reescritura de todas las referencias.

El teléfono del cliente es único en todo el sistema

`clientes.telefono` tiene un `unique` global. Hoy tiene sentido. Con dos negocios significa que **si un cliente va a las dos barberías, la segunda no puede darlo de alta** — o peor, termina compartiendo ficha, historial y tarjeta de fidelidad con el negocio de al lado.

```
alter table public.clientes
  drop constraint clientes_telefono_key,
  add constraint clientes_telefono_por_negocio
    unique (tenant_id, telefono);
```

Lo mismo vale para el `upsert_cliente` que hace la deduplicación por teléfono: hoy busca en toda la tabla.

La configuración es una sola fila

`ajustes_tarjeta` y `ajustes_avisos` son literalmente `check (id = 1)`: **una tarjeta de fidelidad y una regla de avisos para todo el sistema**. Pasan a ser una fila por negocio, con `tenant_id` de llave primaria. Es el cambio más fácil de los tres, y el más fácil de olvidar — porque no falla: simplemente todos los negocios comparten la promoción del primero.

Lo que no es trampa: `reservas_no_overlap` sigue siendo correcto tal como está. Acota por `barbero_id`, y un barbero pertenece a una sede que pertenece a un negocio — el aislamiento le llega heredado.

05 El correo no escala como está

Es la pieza que hoy no se puede clonar ni compartir.

Cinco workflows de n8n atados a este negocio, una credencial SMTP, y **las plantillas viven dentro de n8n** — no en el repositorio. Con un cliente más ya duele: o clonás los cinco por cliente (no escala), o los reescribís.

La forma que aguanta:

- **Las plantillas al repositorio.** Versionadas, revisables, probables. Dos de las cinco ya están en `docs/emails/`.
- **El remitente, en la base.** Una tabla `tenant_correo` con el dominio, el remitente y las credenciales de cada negocio.
- **Un workflow por evento, no por cliente.** La cola ya devuelve el `tenant_id`; el workflow elige remitente y plantilla con ese dato.

A esa altura conviene preguntarse si n8n sigue siendo el lugar. Mandar desde la app con un proveedor transaccional saca un salto del camino crítico y te deja registro de cada correo enviado — que es justo lo que faltó durante los cuatro días de agosto en que el buzón estuvo suspendido.

06 El plan

En orden. Cada fase deja algo servible aunque pares ahí.

FASE 0

Ordenar sin cambiar nada

Sacar las plantillas de correo de n8n. Una sola fuente para la marca (nombre, colores, logo, dominio) en vez de 20 archivos. Matar las 58 referencias a `parque-venezuela` y `plaza-de-la-paz` quemadas en el código — `getSedes()` ya existe. Seguir metiendo reglas en tablas, no en constantes.

Desbloquea: clonar para un segundo cliente pasa de reescribir medio proyecto a cambiar un archivo de marca. Sirve aunque nunca hagas multitenencia.

FASE 1

Las llaves, antes de que haya datos ajenos

`sedes.id` y `servicios.id` a `uuid`, con el slug como columna aparte. Es la migración más invasiva de todo el plan y la única que **se vuelve exponencialmente más cara con cada mes que pasa**.

Desbloquea: que dos negocios puedan tener un servicio llamado igual.

FASE 2

La columna y el relleno

Crear `tenants`, meter a Barbas como el primero, agregar `tenant_id` a las 23 tablas y rellenarlo con ese id. **Sin RLS todavía** — la app sigue funcionando exactamente igual mientras tanto. Índice `(tenant_id, ...)` en las tablas calientes.

Desbloquea: nada visible. Es el andamio, y se puede hacer sin ventana de mantenimiento.

FASE 3

Cerrar la frontera

`tenant_actual()`, el claim en el token, y reescribir las políticas de las 12 tablas sensibles. Arreglar el único del teléfono y los singletons de configuración. **Acá es donde se rompe todo si se rompe**: con un solo negocio en la base, un error de política se ve como "no hay datos", no como una fuga — así que hay que probarlo con dos negocios de mentira antes de crearle.

Desbloquea: el aislamiento real. La base deja de confiar en el código.

Una puerta por negocio

El middleware resuelve el host y de ahí sale el negocio para todo el camino público. Marca, dominio y correo por cliente. El panel deja de ser "el admin" y pasa a ser "el admin de este negocio".

Desbloquea: el segundo cliente entra sin tocar código.

07 Cómo se prueba que aísla

La parte que casi nadie escribe, y la única que atrapa una fuga.

Un error de aislamiento **no falla: devuelve datos de más**. Nadie lo ve hasta que un cliente lo ve. La app ya tiene el molde para esto en `scripts/qa/carreras.mjs`, que dispara carreras reales para comprobar que dos clientes no se quedan con el mismo turno. El equivalente para multitenencia:

- Crear **dos negocios de prueba** con datos propios.
- Con la sesión del negocio A, recorrer **las 23 tablas** y exigir cero filas del negocio B.
- Intentar escribir en una fila de B con la sesión de A — tiene que rebotar, no ignorarse.
- Reservar por el sitio público de A y verificar que la cita quedó en A.
- Correrlo **después de cada migración**, como el de carreras.

Sin esa prueba, la multitenencia es una promesa. Con ella, es una propiedad verificada.

Si mañana hay que decidir

Hacé la **fase 0** igual: es orden, sirve para las dos rutas y no compromete nada. Hacé la **fase 1** antes de firmar el segundo cliente, porque es la única que se encarece sola con el tiempo. Y no toques las fases 2 a 4 hasta que tener varios despliegues te esté doliendo de verdad — la complejidad se paga todos los días, y el aislamiento se paga una vez, pero se cobra caro si sale mal.